SINGAPORE UNIVERSITY OF SOCIAL SCIENCES

ICT 162

Object Oriented Programming

Tutor-Marked Assignment

Jan 2026 Presentation

TUTOR-MARKED ASSIGNMENT (TMA)

This assignment is worth 24% of the final mark for ICT162, Object Oriented Programming.

The cut-off date for this assignment is **Sunday, 22 March 2026, 2355hrs.**

Note to Students:

You are to include the following particulars in your submission: Course Code, Title of the TMA, SUSS PI No., Your Name, and Submission Date.

All TMAs must be submitted electronically through Canvas. No partial submission of TMA will be accepted unless otherwise specified.

Assignment Requirements:

- Unless specified in the question, you CANNOT use packages not covered in this module, e.g., re, collections, numpy, pandas etc.
- All classes and functions must be documented. Provide sufficient comments to your code and ensure that your program adheres to good programming practices such as not using global variables.
- Remember to use dunder __ to represent private instance variables and single underscore _ for protected class variables.
- Do not use the exit() function.
- Failing to do so can incur a penalty of as much as 50% of the mark allotted.

Submission Details:

- Use the template word document provided - SUSS_PI_No-FullName_TMA.docx. Rename the file with your SUSS PI and full name join with "_TMA" e.g., "SUSS_PI-TomTanKinMeng_TMA.docx" (without the quotes).
- Include the following particulars on the first page of the word document, on separate lines: Course Code, SUSS PI No., Your Name, Tutorial Group and Submission Date.
- Copy and paste the source code of each program you write in text format. Submit screenshots for only output of your program, where applicable.
- If you submit the source code as a screenshot, your code will not be marked. That is, screenshot code will be awarded zero mark.
- Submit your solution in the form of a single MS Word document. Do NOT submit as a pdf document. You will be penalised if you fail to submit a word document.
- The word document must be submitted to your respective T group. Besides this, you are required to upload your source code to Vocareum.

Read the following introduction before attempting this TMA.

Welcome to your Pizza House programming adventure!

In this assignment, you'll step into the shoes of a software designer helping a busy pizza company manage its deliveries, orders, and partners.

The challenge is all about **object-oriented programming (OOP)** — you'll design classes, use inheritance and composition, handle exceptions, and bring real-world business rules into code. Think of it as building a mini system that could run a pizza delivery service.

Here's what you'll be tackling:

- **Delivery Partners**: Model both full-time and part-time workers and calculate their salaries based on delivery rounds.
- **Orders & Customers**: Manage pizza orders, track quantities, enforce limits, and handle errors gracefully.
- **Delivery Rounds**: Organize deliveries with capacity rules and connect them back to the partners who earn their pay.
- **Pizza House Management**: Tie everything together to compute salaries and keep the system running smoothly.

By the end, you'll have practiced key OOP principles — **abstraction, encapsulation, inheritance, and polymorphism** — while solving problems that feel practical and fun. It's not just about writing code; it's about thinking like a software architect who can turn a business scenario into a working program.

Question 1 (10 marks)

Question 1a

Let's start with the Customer. Construct the Customer class according to the specification in the class diagram shown below in Figure Q1a.

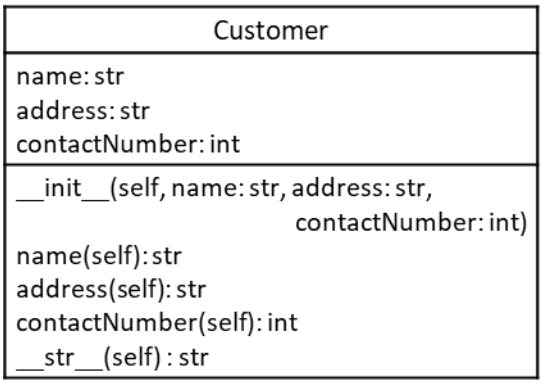

Figure Q1a

Implement the class – Customer

The Customer class has:

- Three instance variables:
  - name (str): name of customer.
  - address (str): address of customer.
  - contactNumber (int): 8 digits Singapore telephone number.
- Constructor performs the following:
  - Initialises these 3 instance variables: name, address and contactNumber., with the given parameters.
- Getter methods for the instance variables: name, address and contactNumber. Use the property decorator.
- The __str__ method returns a string representation of a Customer object, in the following order: name, address, and contact number

```
Name: Kyrie Lim
Address: 94B Bedok North Ave 4, #10-10, Singapore 461094
Contact No: 9877 5877
```

(5 marks)

Question 1b

Write Python class definitions for Pizza. The class diagram in Figure Q1b shows specifications of the Pizza.

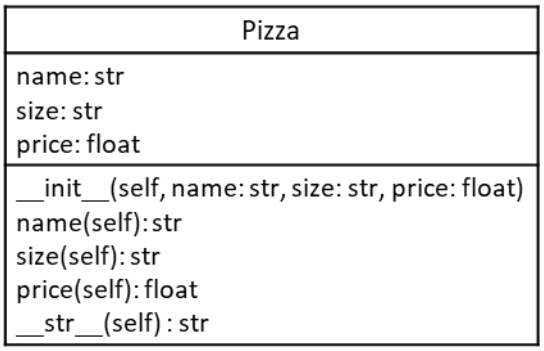

Figure Q1b

The Pizza class has:

- Three instance variables:
  - name (str): name of pizza.
  - size (str): size of the pizza (S, M, L).
  - price (float): price of the pizza based on its size.
- Constructor performs the following:
  - Initialises these 3 instance variables: name, size and price, with the given parameters.
- Getter methods for the instance variables: name, size and price. Use the property decorator.
- The __str__ method returns a string representation of a Pizza object, in the following order: name, size, and price.

```
Name: Hawaiian Plus
Size: M
Price: $9.59
```

(5 marks)

Question 2 (20 marks)

In the Pizza House, an order is placed by a customer. In each order, the customer can select any pizza, of different sizes (S, M, L) and quantity. See a sample order details below, which is self-explanatory:

```
Order ID: 123
Status: Preparing for delivery

Name: Kelly Chan
Address: 331 Marymount Road, #12-08, Singapore 570331
Contact No: 96773211

Hawaiian Plus - M - $  9.59 x 4
Super Supreme - L - $ 12.59 x 3
Total price:        $ 76.13
```

Study the class diagram in Figure Q2, which showcase the relationships between Order, Customer and Pizza.

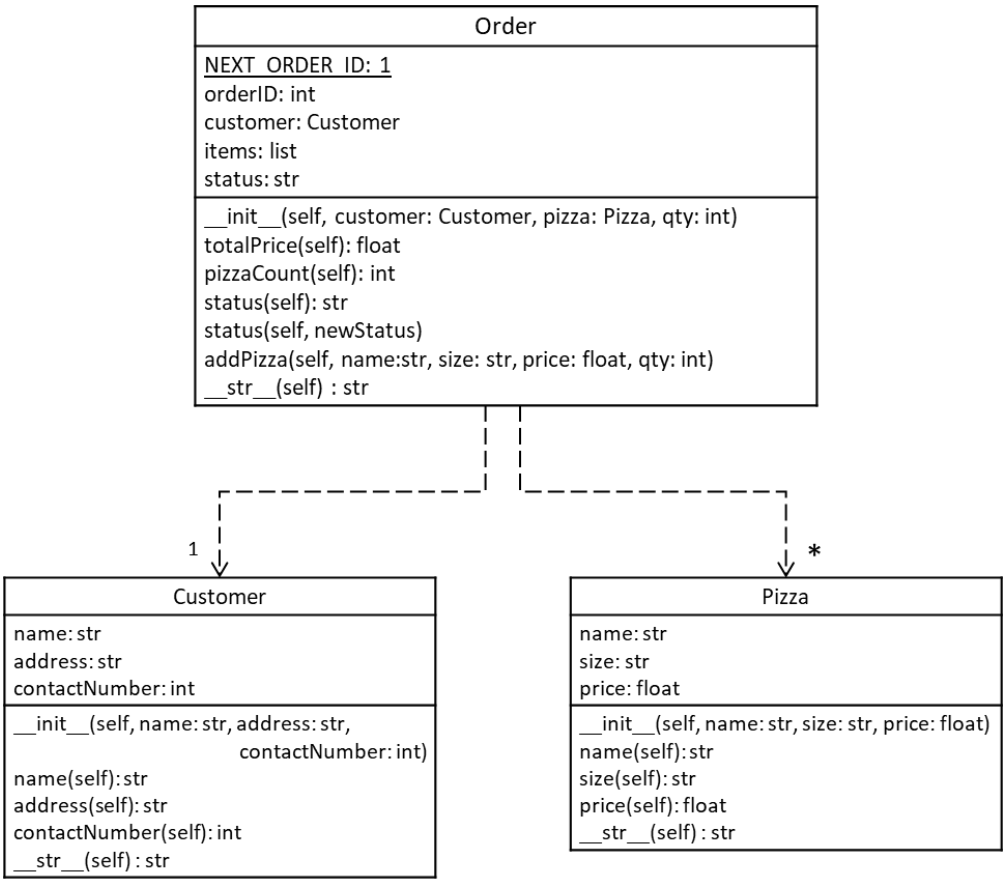

Figure Q2

Question 2a

Implement the Order class.

The Order class has:

- One class variable:
  - NEXT_ORDER_ID: used to generate the running sequence number for orderID, starting from 1.
- Three instance variables:
  - orderID: unique ID to identify an order.
  - customer: represents the customer who has placed this order.
  - items: the pizza(s) that the customer has ordered. This is implemented using a nested list of [pizza, qty].
  - status: the current status of the order (e.g., "Preparing for delivery", "Delivered").
- Constructor that performs the following:
  - Initialise orderID with the class variable NEXT_ORDER_ID, the running sequence number .
  - Set customer to the given parameter customer.
  - First, set items to an empty list. Then, create a list with the given parameter pizza and quantity, i.e., [pizza, qty], and add this list into _items.

- Initialise status to "Preparing for delivery" by default.
- Increment the class variable NEXT_ORDER_ID, so that the next order will not have duplicate orderID.

- Getter and setter method for instance variable status. Use the property decorator.
  - The status setter is to let the Order object update its status with the given parameter newStatus (e.g., from "Preparing for delivery" to "Delivered").
- Getter methods for totalPrice and pizzaCount. Use the property decorator.
  - Property totalPrice will return the current price of the order, computed using all the pizzas' price and quantity in this order.
  - Property pizzaCount will return the number of pizzas in this order.
- The purpose of addPizza method is to add more pizza into this order, i.e., into the collection items.
  - Search through _items to see if there is any existing pizza matching the given parameters name and size.
  - If yes, simply add the parameter quantity to the current quantity.
  - If the search is not successful, this addition is for a brand-new pizza. Create the pizza object, and a list containing the pizza object and quantity. Then add into this order, i.e., into the collection items.
- The __str__ method returns a string representation of an Order object. This is an example in the suggested format:

```
Order ID: 124
Status: Delivered

Name: Kyrie Lim
Address: 94B Bedok North Ave 4, #10-10, Singapore 461094
Contact No: 98775877

Super Supreme - L - $12.59 x 1
Hawaiian Light - M - $8.59 x 2
Total price: $29.77
```

(16 marks)

Question 2b

See following Python codes that create an Order object o1 and has intention to add in more pizzas into this order. Complete the Python codes, with given instructions starting from line 7.

```
1.  def main():
2.      # Creation of Order
3.      o1 = Order( Customer("Herro Tan", "999A, Sentosa Cove", 99123665),
4.                  Pizza("Hawaiian Plus", "M", 9.59), 1)
5.
6.      # write codes to add the following pizzas in this sequence
7.      # 1. Name: Super Supreme, Size: L, Price: 12.59, Qty: 3
8.      # 2. Name: Hawaiian Plus, Size: M, Price: 9.59, Qty: 3
9.      # 3. Name: Chicken Satay, Size: L, Price: 11.59, Qty: 6
10.     #
11.     # Print order details after these additions
12.
```

(4 marks)

Question 3 (20 marks)

In Pizza House, delivery partners are employed to deliver pizza orders to customers. There are 2 types of delivery partners: full-time and part-time. Delivery partners earn their pay based on the number of rounds they deliver in a month. Fees for delivery per round is $32. See Figure Q3a for the fees structure for delivery partners.

Partner Type	Number of Rounds Delivered (r)	
	r <= 30	r > 30
Full-time	basic salary + r * f	basic salary + r * f
Part-time	r * f	30 * f + (r-30) * (f*1.85)
		where f represents the fees per round.

Figure Q3a

The class diagram in Figure Q3b shows specifications of the delivery partners.

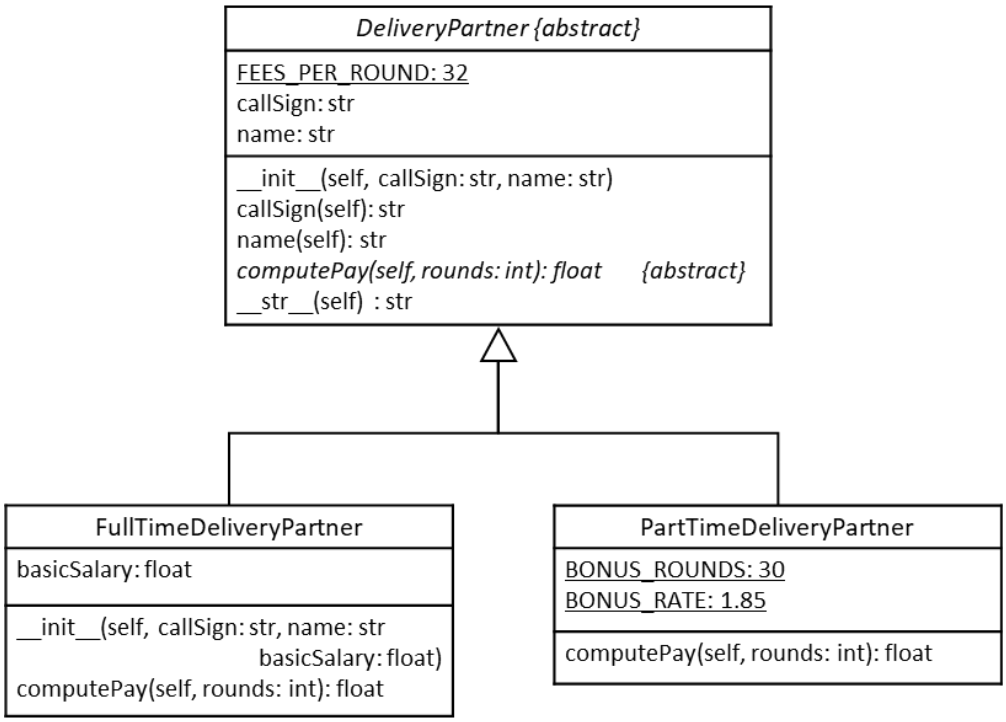

Figure Q3b

Question 3a

Implement the abstract class – DeliveryPartner.

- FEES_PER_ROUND: defines the fees earned delivering a round.
- It has 2 instance variables:
  - callSign (str): the preferred alias or code for this delivery partner. E.g., "DP-007", "Maverick"
  - name (str): name of the delivery partner.
- Constructor initialises these 2 instance variables: callSign and name.
- Getter methods for callSign and name. Use the property decorator.

- One abstract method, computePay() which returns the computed pay for the DeliveryPartner, given the number of rounds delivered.
- The __str__ method returns a string representation of a DeliveryPartner object, which should include the name and callSign. This is an example in the suggested format:

```
Name: Lim Lim Kee      CallSign: DP-007
```

(6 marks)

Question 3b

Write Python class definitions for FullTimeDeliveryPartner and PartTimeDeliveryPartner class.

The FullTimeDeliveryPartner class is a subclass of DeliveryPartner, and it has:
- One additional instance variable:
  - basicSalary (float): monthly basic salary of a full-time delivery partner.
- Implement the method computePay() based on the fees structure given in Figure Q3a.

The PartTimeDeliveryPartner class is a subclass of DeliveryPartner, and it has:
- Two class variables:
  - BONUS_ROUNDS: beyond this round number, a part-time delivery partner will be rewarded with a higher fees per round.
  - BONUS_RATE: representing the bonus rate on the fees per round, should the part-time delivery partner qualify.
- Implement the method computePay() based on the fees structure given in Figure Q3a.

(10 marks)

Question 3c

Write Python statements to create the appropriate DeliveryPartner objects using the information below, and to print their pay.

Name	Call Sign	Basic Salary	Rounds Delivered
Lim Lim Kee	DP-007	-	50
Tom Yam Kong	Maverick	1,200	40

(4 marks)

Question 4 (25 marks)

Apply the principles of object-oriented programming and develop a program to help Pizza House manage their "DeliveryRound" and implement method to compute the salary of Delivery Partners. Study the class diagram in Figure Q4, which now features the DeliveryRound class.

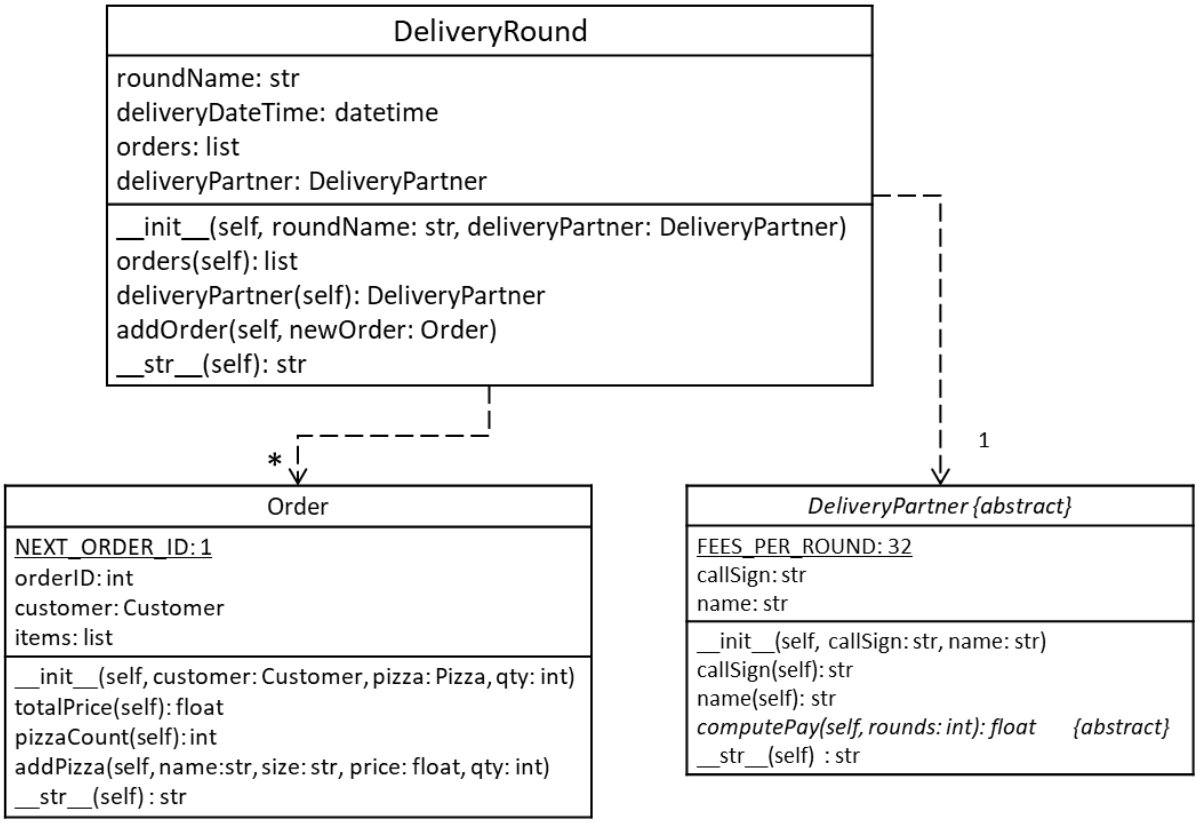

Figure Q4

In Pizza House, each Delivery Partner is given a vehicle that has storage capacity of 12 Pizzas. Hence in a delivery round, the maximum number of pizzas the delivery partner can deliver is 12.

Question 4a

Define the class OrderException, which is a subclass of the Exception class. This class has no additional attribute or method.

(1 mark)

Question 4b

You are given the following class that has one class variable:

```
class Config:
    _MAX_PIZZA = 12
```

- MAX_PIZZA: represents the maximum number of pizzas or the storage capacity of the vehicle used to deliver pizzas. In a possible scenario, the delivery partner delivers one order up to 12 pizzas.

Enhance the addPizza method of the Order class to include exception handling as follows:

- If this addition results in items having more pizzas than MAX_PIZZA, the pizza(s) is not added into the items, raise OrderException with appropriate message.

(4 marks)

Question 4c

Implement the DeliveryRound class.

The DeliveryRound class represents an actual delivery round that the Pizza House will setup for a delivery partner. It has the following attributes:

- Four instance variables:
  - roundName represents the name given for this delivery round. E.g., "East Coast Path-1".
  - deliveryDateTime is the start time of the delivery round.
  - orders is a list that represents all the orders for this delivery round.
  - deliveryPartner represents the delivery partner doing this delivery round.
- The constructor performs the following:
  - Initialise roundName and deliveryPartner with the given parameter roundName and deliveryPartner.
  - Set deliveryDateTime to the current datetime, and _orders to an empty collection.
- Getter methods for orders and deliveryPartner. Use the property decorator.
- The addOrder method has newOrder (an Order object) as parameter.
  - If this addition results in orders having more pizzas than 12, i.e., this DeliveryRound has more than 12 pizzas, raise OrderException with appropriate message. Hence, the newOrder is not added into this delivery round.
  - Otherwise, add the newOrder into the collection orders.
- The __str__ method returns a string representation of a DeliveryRound object. You can define the output layout for the delivery round, as long as all details are presented.

(17 marks)

Question 4d

Referring to Figure Q4, for the following pairs of classes, identify between object composition and inheritance, and justify your choices:

- DeliveryRound and Order
- DeliveryRound and FullTimeDeliveryPartner
- DeliveryRound and Pizza

(3 marks)

Question 5 (25 marks)

In this question, you will implement the **PizzaHouse** class to manage delivery partners, orders, and delivery rounds. The focus is setting up the DeliveryRound correctly. The full class diagram for Pizza House is provided in Figure Q5.

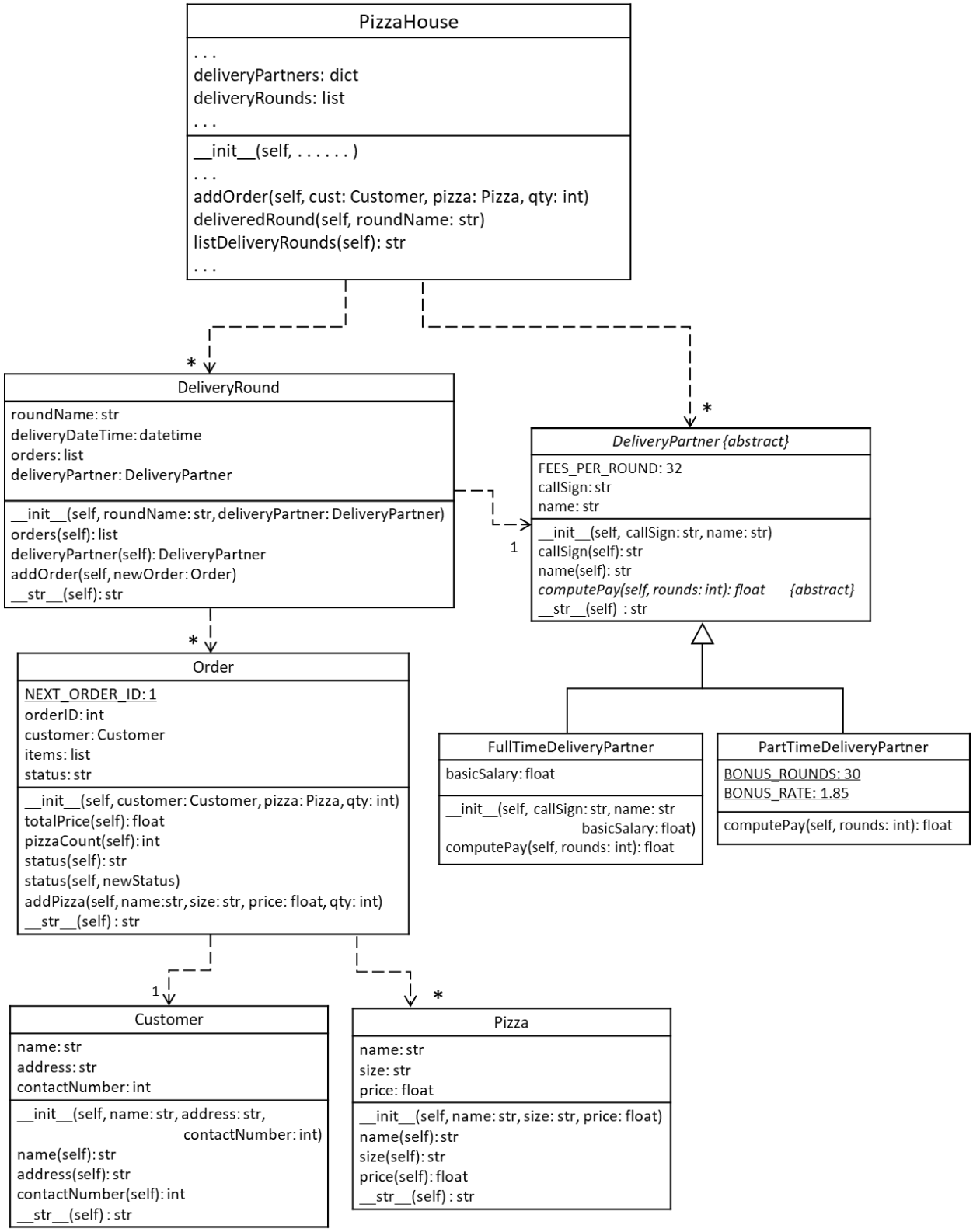

Figure Q5

Question 5a

Specifications of PizzaHouse class:

- Two instance variables:
  - deliveryPartners: a dictionary containing all delivery partners of Pizza House.
    - The key is the partner's call sign.
    - The value is another dictionary with the following fields:
    - "partner" → the DeliveryPartner object.
    - "assigned" → a boolean indicating whether the partner is currently assigned to a delivery round.
    - "rounds" → an integer tracking the number of rounds this partner has delivered.
  - deliveryRounds: a list containing all delivery rounds created by Pizza House.
  - *(you can add more instance variables, provided they make sense)*
- Constructor:
  - Initialises both lists as empty.
  - Reads delivery partner data from a file provided in Appendix A.
  - Creates the appropriate FullTimeDeliveryPartner or PartTimeDeliveryPartner objects and stores them in deliveryPartners.
- Methods to implement:
  - addOrder(customer, pizza, qty):
    - Creates a new Order object.
    - Checks if this order can be added into an existing DeliveryRound (without exceeding the maximum capacity of 12 pizzas).
    - If yes, add the order into that round.
    - If no existing delivery round can accommodate it, create a new DeliveryRound, assign a delivery partner (choose one unassigned delivery partner from deliveryPartners, otherwise, raise exception), and add the order into this new round.
    - Update the partner's "assigned" flag.
  - deliveredRound(roundName):
    - Parameter roundName represent the name of the delivery round to be marked as delivered.
    - Search for the delivery round with the given roundName.
    - If not found, raise an OrderException("Delivery round not found.").
    - For each order in that round, update its status to "Delivered".
    - Mark the delivery partner as unassigned in deliveryPartners.
    - Increment the partner's "rounds" counter (tracking completed rounds).
    - In addition, raise OrderException if the round does not exist, or if the round has already been delivered (to prevent double-counting).
  - listDeliveryRounds():
    - Displays details of all delivery rounds in PizzaHouse.
    - Each delivery round should show:
      - Round name
      - Delivery partner (name and callSign)
      - Orders in the delivery round (pizza name, size, price, qty)
      - Total pizzas in the delivery round

(17 marks)

Question 5b

For this question, include a main() program that:

- Create the PizzaHouse object.
- Add in at least 10 orders that demonstrate the logic you implemented in PizzaHouse class:
  - Orders can be hardcoded or interactively entered.
  - When orders are added, they should be grouped into delivery rounds of up to 12 pizzas each.
  - If a round is full, a new round is created and assigned to the next unassigned delivery partner.
- Invoke the listDeliveryRound() method to display all delivery rounds and their details clearly.
- Update some delivery rounds to "Delivered", some bogus delivery round etc.
  - You should expect that the delivery rounds are "Delivered" and the delivery partners are also "Unassigned", with "rounds" counter incremented.
- Invoke again listDeliveryRound() method to display all delivery rounds to show the differences.
- Compute and print the pay for all the delivery partners.
- Remember to manage all exceptions including input error (if any).

(8 marks)

Appendix A

List of Delivery Partners

Name	Call Sign	Basic Salary	Type
Lim Lim Kee	DP-007	-	Part-time
Tom Yam Kong	Maverick	1,200	Full-time
Sarah Tan	SwiftRider	-	Part-time
Rajesh Kumar	IronHorse	1,500	Full-time
Mei Ling Wong	StarDrop	-	Part-time
Ahmad Hassan	FalconX	1,350	Full-time
Jessica Lee	NightOwl	-	Part-time
Daniel Chua	ThunderBolt	1,250	Full-time
Nurul Aisyah	Breeze	-	Part-time
Kelvin Wong	RoadRunner	1,400	Full-time
Sally Tan	SpeedRider	-	Part-time

Hint:

- Format the above into a text file with columns: Name, Call Sign, Basic Salary, Type.

---- END OF ASSIGNMENT ----